

Encapsulation with Objects

Unlike variables used in regular functions, instance variables in objects persist across different method calls. To keep the remaining discussion clear, I'll introduce a new term: *client*. (I don't want to use the term *user* here since that typically refers to the human user of the final program.)

client

Any software that creates an object from a class and makes calls to the methods of that object.

We must also consider the duality of *inside* versus *outside* an object or class. When you're working inside a class (writing the code of the methods in a class), you need to concern yourself with how the different methods of the class share the instance variables. You consider the efficiency of your algorithms. You think about what the interface should look like: what methods you should provide, what the parameters for each are, and what should be used as default values. In short, you are concerned with the design and implementation of the methods.

From the outside, as a client programmer, you need to know the interface of the class. You're concerned with what the class methods do, what arguments should be passed in, and what data is passed back from each method.

A class therefore provides encapsulation by:

- Hiding all details of implementation in its methods and instance variables
- Providing all the functionality a client needs from an object through its interface (the methods defined in the class)

Objects Own Their Data

In object-oriented programming, we say that the data inside an object is *owned* by the object. OOP programmers generally agree that, as a good